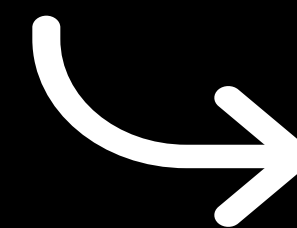


Dominando SubViewports en Godot

Podés acceder al material
de trabajo escaneando el qr



<https://github.com/chulogiamportone/ViewportTests>

01 SubViewport

El SubViewport es un nodo que crea un Framebuffer independiente.

Es un contexto de renderizado aislado del árbol principal de la escena.

Por sí mismo, el ***SubViewport es invisible***; no tiene representación gráfica en la pantalla final hasta que su contenido es proyectado sobre una superficie (UI o Textura)

Características Principales:

- **Aislamiento:** Todo lo que ocurre dentro de él (cámaras, luces, objetos) se procesa en un paso de renderizado separado.
- **Resolution (Size):** Tiene una resolución fija en píxeles (ej. 512×512). Esta resolución es independiente de la resolución de la ventana del juego.
- **Render Target:** El resultado visual del SubViewport se almacena en la memoria de video (VRAM) como una textura dinámica (ViewportTexture).

Tipos de Implementación

UI Composite

Renderizado directo en el árbol de Control
(SubViewportContainer)

Proyección de cámara secundaria sobre capa 2D
(CanvasLayer).

Estructura de Nodos:

- **CanvasLayer** (Aislamiento de la cámara principal)
 - SubViewportContainer (Nodo de Control: Maneja layout y anchors)
 - **SubViewport** (Contexto de render)
 - **Camera3D** (Proyección Ortogonal/Perspectiva)

Texture Resource

Renderizado a una textura muestreable en materiales (ViewportTexture)

Muestreo de Render Target en superficies 3D
(MeshInstance3D).

Workflow de Materiales:

1. **Nodo:** SubViewport independiente (Sin Container, suele agruparse en un Node3D contenedor).
2. **Target:** MeshInstance3D (Monitor/Pantalla).
3. **Resource:** StandardMaterial3D > Albedo > Texture > New ViewportTexture.



Sub Viewport Container

UI Composite

Caso de Uso A

Renderizado en UI (Mini-mapa)

Proyección de cámara secundaria sobre capa 2D (CanvasLayer)

Estructura de Nodos:

- **CanvasLayer** (Aislamiento de la cámara principal)
 - **SubViewportContainer** (Nodo de Control: Maneja layout y anchors)
 - **SubViewport** (Contexto de render)
 - **Camera3D** (Proyección Ortogonal/Perspectiva)

Configuración Clave:

- **Stretch:** True (En el Container). Escala el buffer interno al tamaño del rect UI.
- **Projection:** Orthogonal (Recomendado para mapas técnicos/ GPS).
- **Size:** Definir resolución fija (ej. 512px) para evitar aliasing en el downscaling.



MeshInstance3D

Viewport Texture



Caso de Uso B

ViewportTexture y MeshInstance3D

Muestreo de Render Target en superficies 3D (MeshInstance3D)

Workflow de Materiales:

1. **Nodo:** SubViewport independiente (Sin Container, suele agruparse en un Node3D contenedor).
2. **Target:** MeshInstance3D (Monitor/Pantalla).
3. **Resource:** StandardMaterial3D > Albedo > Texture > New ViewportTexture.

Dato Crítico:

- **Local to Scene:** True.
 - Si se instancia la escena múltiples veces (ej. varios monitores), *Local to Scene permite que cada material apunte a su propio Viewport dinámico*. Sin esto, todos los monitores mostrarán el render del último viewport cargado.
- **Render Target. Update_mode=WHEN_VISIBLE**
 - *Detiene el render* si la vision del objeto no está en los parametros de la cámara principal.



OTRAS FORMAS DE USAR SUBVIEWPORTS

1. Portales: Conexión visual fluida entre dos puntos dispares del mapa.



2. Control Remoto (Drones/Misiles): Picture-in-Picture para guiar un proyectil mientras ves a tu personaje cubrirse en la cámara principal.



3. Rayos X / Escáner: Un recuadro que, al pasarlo sobre un enemigo, revela su esqueleto o puntos débiles.



Gracias

<https://giampdev.itch.io/>

<https://github.com/chulogiamportone>

<https://www.linkedin.com/in/juliangiamportone/>
